Functions

A function is a block of code that performs a specific task. We can call the function whenever we need that task instead of writing the same code repeatedly.

Function Declaration, Definition and Calling

Function Declaration Tells the compiler the function name, parameters and return type int add(int, int);
Function Definition Contains the actual code to perform the task. int add(int a, int b) { return a + b; }
Function Calling Executes the function. result = add(10, 20);

```
#include <stdio.h>
int add(int, int);
int main()
{
    int result;
    result = add(10, 20);
    printf("Sum = %d", result);
    return 0;
}
int add(int a, int b)
{
    return a + b;
}
```

Sum = 30

Function Parameters and Return Types

Parameters receive values from the calling function. The return type specifies what kind of value the function sends back.

No parameters and no return value

```
#include <stdio.h>
void display()
{
    printf("Welcome to C Programming");
}
int main()
{
```

```
    display();
    return 0;
}
```
Welcome to C Programming

Parameters but no return value

```
#include <stdio.h>
void sum(int a, int b)
{
    printf("Sum = %d", a + b);
}
int main()
{
    sum(10, 20);
    return 0;
}
```
Sum = 30

No parameters but returns a value

```
#include <stdio.h>
int getNumber()
{
    return 100;
}
int main()
{
    int n = getNumber();

    printf("%d", n);

    return 0;
}
```
100

Parameters and a return value

```
#include <stdio.h>
int multiply(int a, int b)
{
    return a * b;
}
int main()
{
    int result = multiply(5, 4);

    printf("Product = %d", result);

    return 0;
}
```

```
Product = 20
```

Call by Value

In call by value, a copy of the actual argument is passed to the function.

Changes made to the parameter inside the function do not affect the original variable.

```
#include <stdio.h>
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
int main()
{
    int x = 10, y = 20;
    printf("Before swapping %d %d",x,y);
    swap(x, y);
    printf("\nAfter swapping %d %d",x,y);
    return 0;
}
Before swapping 10 20
After swapping 10 20
```

Pointer

A pointer is a variable that stores the memory address of another variable

```
#include <stdio.h>
int main()
{
    int x = 10;
    int *p = &x;
    printf("%d\n", x);
    printf("%d\n", *p);
    printf("%p\n", p);
    return 0;
}
10
10
0xffffd9029224
```

Call by Reference

In pass by reference, the function receives the address of the variable instead of a copy.

The function can directly modify the original variable in the caller.

```
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
int main()
{
    int x = 10, y = 20;
    printf("Before swapping %d %d",x,y);
    swap(&x, &y);
    printf("\nAfter swapping %d %d",x,y);
    return 0;
}
```

Before swapping 10 20
After swapping 20 10

Nesting of Functions

Nesting of functions means calling one function inside another function

```
#include <stdio.h>
void display()
{
    printf("Hello Students!\n");
}
void message()
{
    display();  // Calling display() inside message()
    printf("Welcome to C Programming!\n");
}
int main()
{
    message();
    return 0;
}
```

Hello Students!
Welcome to C Programming!

Recursive Functions

A recursive function is a function that calls itself.

Every correct recursive solution needs two important parts:

- Base condition: Stops recursion.
- Recursive call: Calls the same function with a smaller or simpler problem.

Factorial using recursion

```
#include <stdio.h>
int factorial(int n)
{
    if (n == 0|| n==1)
        return 1;
    else
    return n * factorial(n - 1);
}
int main()
{
    printf("Factorial = %d", factorial(5));
    return 0;
}
Factorial = 120
```

Sum of numbers using recursion

```
#include <stdio.h>
int sum(int n)
{
    if (n == 1)
        return 1;
    return n + sum(n - 1);
}
int main()
{
    printf("Sum = %d", sum(5));
    return 0;
}
Sum = 15
```

Fibonacci sequence using recursion

```
#include <stdio.h>
int fib(int n)
{
   if (n == 0 || n == 1)
      return n;
   else
   return fib(n - 1) + fib(n - 2);
}

int main()
{
   int i;
   for (i = 0; i < 7; i++)
      printf("%d ", fib(i));
   return 0;
}
```

0 1 1 2 3 5 8

Scope and Lifetime of Variables

Scope refers to the part of a program where a variable can be accessed or used.

Lifetime refers to the period during which a variable exists in memory while the program executes.

Local Variable

A variable declared inside a function is called a local variable. It can be used only inside that function.

```
#include <stdio.h>
void display()
{
   int a = 10;
   printf("%d", a);
}

int main()
{
   display();
   return 0;
}
```

10

Block Scope variable

A variable declared inside a block { } can be used only inside that block.

```
#include <stdio.h>
int main()
{
    {
        int a = 20;
        printf("%d\n", a);
    }

    return 0;
}
20
```

Static Variable

A static local variable exists throughout the entire program execution. Its value is retained between function calls.

```
#include <stdio.h>
void display()
{
    static int a = 0;
    a++;
    printf("%d\n", a);
}

int main()
{
    display();
    display();
    display();
    return 0;
}
1
2
3
```

Global Variable

A variable declared outside all functions is called a global variable. It can be accessed by functions declared after it, subject to normal declaration rules.

```
#include <stdio.h>
int a = 10;
void display()
{
```

```
    printf("%d\n", a);
}

int main()
{
    printf("%d\n", a);
    display();
    return 0;
}
10
10
```

Header Files in C

A header file contains declarations that can be shared between C source files.

Header files commonly have the .h extension.

Header file	Purpose
`stdio.h`	Input and output functions such as `printf()` and `scanf()`
`string.h`	String functions such as `strlen()` and `strcpy()`
`math.h`	Mathematical functions such as `sqrt()` and `pow()`
`stdlib.h`	Utility functions such as `malloc()` and `exit()`

Using Standard Header File

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>

int main()
{
    char name[] = "Ashok";

    printf("Name: %s\n", name);
    printf("Length: %d\n", strlen(name));
    printf("Square root of 25 = %.1f\n", sqrt(25));
    printf("Absolute value of -10 = %d\n", abs(-10));

    return 0;
}
```

```
Name: Ashok
Length: 5
Square root of 25 = 5.0
Absolute value of -10 = 10
```

Modular Programming

Modular programming means dividing a large program into smaller, manageable parts called modules.

Addition Using Two Files

File 1: addition.h

```
int add(int a, int b);
```

File 2: addition.c

```
#include "addition.h"

int add(int a, int b)
{
    return a + b;
}
```

File 3: main.c

```
#include <stdio.h>
#include "addition.h"

int main()
{
    int result = add(10, 20);
    printf("%d", result);
    return 0;
}
30
```

Advantages of Modular Programming

- **Easy to understand:** Each module performs a specific task.
- **Code reusability:** The same function can be used by different programs.
- **Easy debugging:** Errors can be located more easily.
- **Easy maintenance:** One module can be modified without rewriting the entire program.
- **Teamwork:** Different programmers can work on different modules.